MLPC 2026 Task 4: Data Classification

Team Unified Decoders

Alexandre Escudier Agostinho **Jakob Hinterholzer**

1 Dataset Preparation

1.1 Label Aggregation

The raw annotations have shape $[T, C, A]$, where T is the number of time segments, $C = 15$ the number of sound event classes, and A the number of annotators. Each value represents the proportional temporal overlap between an annotation event and a segment, ranging from 0.0 (no overlap) to 1.0 (full overlap). Our goal is to collapse the annotator dimension to produce binary training labels of shape $[T, C]$.

We applied a two-step **majority voting** strategy. First, each annotator's overlap values are binarized using a threshold of 0.5: values ≥ 0.5 are treated as positive and values below as negative. Second, the binarized values are averaged across annotators, and a majority threshold of 0.5 is applied: if at least half of the annotators marked a segment as active for a given class, the final label is set to 1.

Majority voting is well-suited for this task because it is robust to individual annotator noise. If one annotator mislabels a segment, the majority consensus corrects the error, producing cleaner training labels. It also yields hard binary labels compatible with standard classifiers. A potential limitation is that rare or ambiguous events marked by only a single annotator are discarded. Alternative strategies include union-based aggregation (label as positive if *any* annotator agrees), which increases recall at the cost of noisier labels, and soft labels (keeping the continuous average), which preserves annotator disagreement but requires classifiers that support probabilistic targets. We compared all three strategies and found majority voting to provide the most balanced label distribution across classes.

1.2 Data Split

We split the data into training (70%), validation (15%), and test (15%) sets, resulting in 117,600 training segments, 25,249 validation segments, and 25,390 test segments.

1.2.1 Information Leakage Prevention

The split is performed at the **collector level**: all recordings from the same collector are assigned to the same split. This prevents two forms of information leakage. First, segments from the same recording share background noise, room acoustics, and microphone characteristics. Placing them in different splits would allow the model to exploit these correlations rather than learning genuine acoustic patterns. Second, recordings from the same collector typically share the same room, device, and ambient conditions. Keeping all of a collector's files together prevents the model from memorizing collector-specific acoustic signatures.

1.2.2 Class Distribution Preservation

Maintaining consistent class distributions across splits ensures that validation and test metrics reflect real performance rather than sampling artifacts. We used a greedy assignment strategy: collectors are assigned one by one to whichever split is furthest below its target segment count, which approximately balances both the total number of segments and the per-class frequencies. Figure 1 shows the resulting class frequency distributions across splits.

1.3 Preprocessing

We apply the following preprocessing pipeline, fit exclusively on the training set to prevent data leakage:

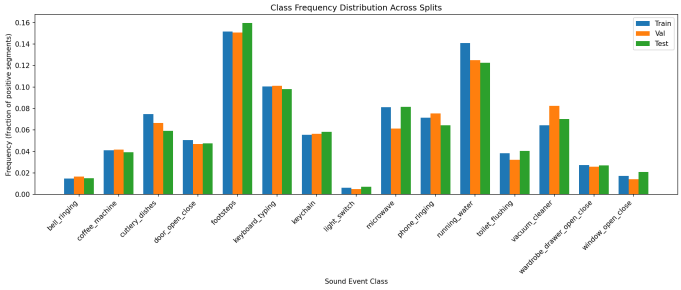

Figure 1: Per-class frequency distributions across train, validation, and test splits. The greedy collector-level assignment produces approximately consistent distributions.

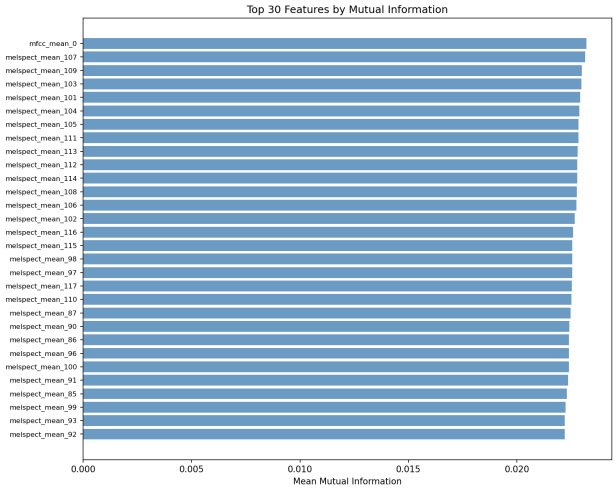

Figure 2: Top features ranked by mean mutual information across all 15 classes.

1) Invalid value handling. Some acoustic features can produce NaN or Inf values (e.g., spectral flatness on near-silent segments). We checked for non-finite values and found none in our dataset, but included this step for robustness.

2) Low-variance feature removal. Features with near-zero variance ($< 10^{-8}$) carry no discriminative information. No features were removed by this criterion, confirming that all 480 features (28 feature groups $\times$ 4 aggregation statistics $\times$ varying dimensionalities) contain meaningful variation.

3) Standardization (z-score normalization). Each feature is transformed to zero mean and unit variance. This is essential because the acoustic features span vastly different numerical ranges: feature means range from -79.0 to $5{,}066.4$, and standard deviations from 0.002 to $1{,}337.6$. Without normalization, high-magnitude features (e.g., spectral centroid, bandwidth) would dominate distance-based and gradient-based classifiers. The scaler is fit on training data only.

4) Mutual information feature selection. We rank all 480 features by their mutual information (MI) with each of the 15 class labels, average the MI scores across classes, and retain the top 50% (240 features). This reduces dimensionality, removes irrelevant features, and can improve generalization. The top-ranked features are the first MFCC coefficient (representing overall energy) and high-frequency mel spectrogram bands (indices 101–113), indicating that spectral energy distribution is most informative for distinguishing sound events. Figure 2 shows the feature importance ranking.

2 Evaluation

2.1 Evaluation Metric

We selected **macro-averaged F1 score** as the primary evaluation metric. F1 is the harmonic mean of precision and recall, rewarding classifiers that both detect events (high recall) and avoid false alarms (high precision).

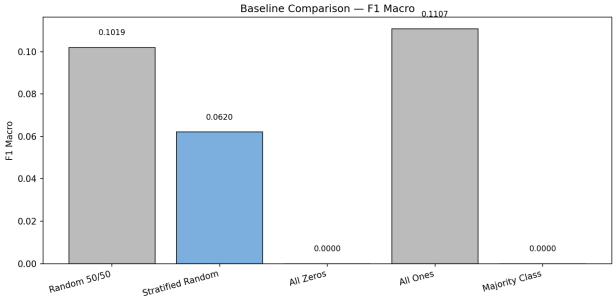

Figure 3: Comparison of baseline strategies. The stratified random baseline (highlighted) serves as the primary reference for classifier evaluation.

Accuracy is unsuitable because the dataset is highly imbalanced: most segments have no active sound event for most classes. A trivial all-zeros classifier would achieve high accuracy but detect nothing. Macro-averaging computes F1 independently per class and takes the unweighted mean, giving equal importance to every sound class regardless of frequency. This is preferable to micro-averaging, which would be dominated by frequent classes like footsteps, potentially hiding poor performance on rare but important classes such as light_switch or bell_ringing. Macro F1 ensures that rare classes are not neglected. We additionally reported per-class F1 and micro F1 as complementary metrics.

2.2 Baseline Performance and Upper Bound

Our primary baseline is a **stratified random classifier** that predicts each class as active with probability equal to its empirical frequency in the training set, without using any features. This baseline achieves a macro F1 of **0.060** on the test set, confirming that class frequency information alone is insufficient for sound event detection.

Perfect performance (F1 = 1.0) is unlikely to be achievable for several reasons: (1) inter-annotator disagreement introduces inherent label noise that even a perfect model cannot resolve; (2) the 1-second aggregation windows with 0.5 s hop size blur event boundaries, making segments at onsets and offsets ambiguous; (3) the acoustic features are summary statistics that may not capture all discriminative information; and (4) some sound classes can be acoustically similar and difficult to distinguish. A realistic upper bound is estimated in the range of F1 ≈ 0.7–0.9 depending on the class, with acoustically distinctive sustained sounds (e.g., vacuum cleaner, running water) being easier to detect than short transient events (e.g., light switch, window opening).

3 Experiments

We compare two classifiers from different model classes: a linear Support Vector Machine (one-vs-rest `LinearSVC`) and CatBoost gradient-boosted trees (one binary model per class). To keep the comparison fair, both use the identical collector-level 70/15/15 split (seed 42, so no collector appears in two folds and recording-style leakage is impossible), the same 480-dimensional mean+std feature set, per-class decision thresholds tuned on the validation set to maximize F1, and class weighting to counter the strong imbalance. The SVM additionally z-score standardizes the features (statistics fit on train only), since linear models are scale-sensitive whereas trees are not. Hyperparameters and the final model are selected by *macro*-F1, the metric justified in Section 2 for this imbalanced multi-label task, and both are compared to the stratified-random baseline of Section 2.

SVM (C). The single most important `LinearSVC` hyperparameter is the inverse regularization strength C: small C enforces a wider margin (more bias), large C fits the training data harder (more variance). Validation macro-F1 (Figure 4) peaks at $C = 0.001$ (0.473) and is slightly lower for both stronger (0.0001: 0.469) and weaker (0.01: 0.466) regularization; an initial wider sweep showed larger C (≥ 0.1) to be both worse and disproportionately slower to fit, so we focused on the strongly-regularized regime. Tuning the per-class thresholds is essential, raising the SVM from 0.386 at the default decision boundary to 0.480 macro-F1.

CatBoost (depth, learning rate, L2). We vary the three most influential knobs one at a time around the defaults (Figure 5). Tree *depth* controls interaction order and peaks at 6 (shallower underfits, deeper overfits); the *learning rate* is best at 0.1 (0.2 overshoots and collapses to 0.521); and a slightly stronger *L2 leaf regularization* of 6 beats the default 3. The curves are otherwise flat (all within ≈ 0.006 macro-F1), so CatBoost is robust to these settings; we select `depth=6, learning_rate=0.1, l2_leaf_reg=6`.

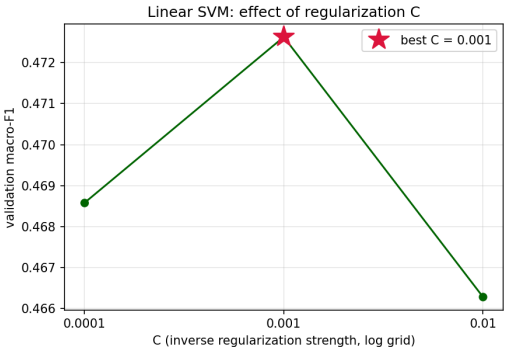

Figure 4: Linear SVM: validation macro-F1 vs. regularization C (log grid), best at $C = 0.001$.

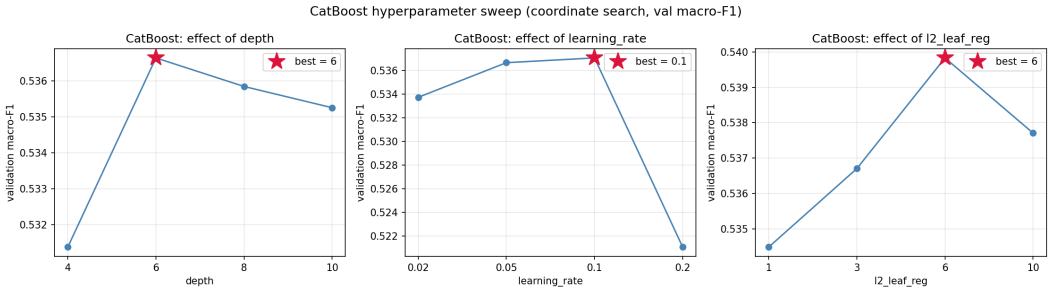

Figure 5: CatBoost coordinate sweep: validation macro-F1 vs. each hyperparameter (others held at default). Selected values starred.

Comparison. Table 1 reports test performance with the selected hyperparameters. Both models hugely beat the baseline (0.063 macro-F1): the SVM reaches 0.480 and CatBoost 0.535, an $8.5\times$ improvement. Threshold tuning is the single biggest post-processing gain for both (SVM $+0.094$, CatBoost $+0.064$). CatBoost is the stronger model by $+0.056$ macro-F1. The per-class breakdown (Table 2) explains why: both models agree on which classes are easy (sustained, spectrally distinctive sources like *running_water*, *keyboard_typing*, *vacuum_cleaner*) and which are hard (rare, transient "open/close" events like *light_switch*, *wardrobe*, *window*), so the difficulty is intrinsic to the data rather than the model. CatBoost's advantage is largest exactly where nonlinear feature interactions help — *vacuum_cleaner* ($+0.12$), *toilet_flushing* ($+0.11$), *keychain* ($+0.09$) — which the linear SVM cannot represent. The only class where the SVM edges ahead is *phone_ringing*, and *bell_ringing* is a tie. CatBoost's practical cost is training one model per class; the SVM is conceptually simpler but never competitive and slow at large C.

Table 1: Test performance (tuned thresholds unless noted); identical split, features and baseline.

Model	$F1_{ma}$	$F1_{mi}$
Baseline (random)	0.063	0.090
SVM (default thr.)	0.386	0.416
SVM (tuned thr.)	0.480	0.559
CatBoost (thr. 0.5)	0.471	0.515
CatBoost (tuned)	**0.535**	**0.616**

Table 2: Per-class test F1 (tuned), sorted by CatBoost.

Class	SVM	CatBoost
running_water	0.74	**0.77**
keyboard_typing	0.72	**0.76**
vacuum_cleaner	0.61	**0.73**
cutlery_dishes	0.55	**0.63**
microwave	0.57	**0.61**
phone_ringing	**0.64**	0.61
keychain	0.51	**0.60**
coffee_machine	0.56	**0.58**
footsteps	0.52	**0.58**
toilet_flushing	0.47	**0.58**
bell_ringing	0.46	0.46
door_open_close	0.33	**0.39**
window_open_close	0.22	**0.31**
wardrobe_drawer	0.20	**0.26**
light_switch	0.09	**0.15**

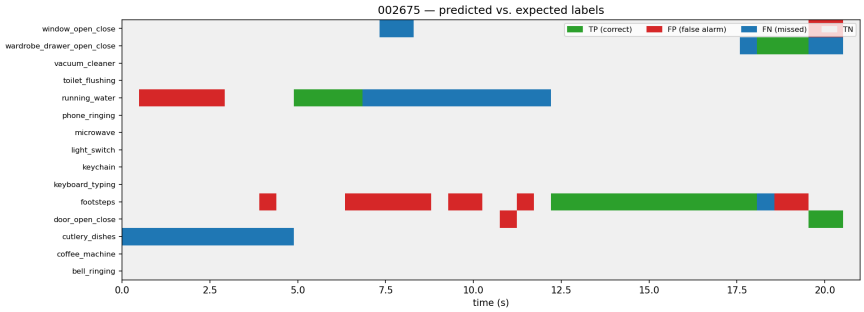

Figure 6: File 002675 – predicted vs. expected labels per class over time (green=correct, red=false alarm, blue=missed, grey=true negative).

4 Case Study and Reflection

We qualitatively inspect the tuned CatBoost detector on two test recordings (collectors unseen in training). For each we show a per-segment timeline of the 15 classes over time, overlaying predicted vs. expected labels coloured as true positive (correct detection), false positive (false alarm), false negative (missed), or true negative.

File 002675 (Figure 6, 43 segments, 6 classes) is a busy scene. The central *footsteps* block (12.5–17.5 s) is detected almost perfectly, but *footsteps* also fires repeatedly in earlier segments where it is not annotated — high recall, poor precision for this impulsive, broadband class. *running_water* shows the opposite failure: a false alarm at the start, one correct segment, then a long missed stretch, i.e. the detector latches onto the onset but loses the sustained part. *cutlery_dishes* (10 segments) is missed entirely, and *door_open_close* is correctly caught near the end.

File 000118 (Figure 7, 34 segments, 5 classes) is dominated by a vacuum cleaner that is largely missed (15 of 19 segments are false negatives) despite its strong overall F1 (0.73): the model only catches the loudest central portion, suggesting it keys on intensity rather than a stable spectral signature, so the quieter onset and tail are lost. *running_water* at the start is detected cleanly, while *footsteps* again mixes a correct block with scattered false alarms.

Reflection. Per-class test F1 ranges from 0.77 (*running_water*) to 0.15 (*light_switch*). The reliable classes are sustained, spectrally rich sources; the hardest are rare, brief "open/close" events that are both heavily under-represented and acoustically transient with no characteristic spectrum. The cross-class confusion matrix (Figure 7, right) shows the errors are perceptually plausible rather than random: *bell*↔*phone* (both tonal ringing), *door*↔*window* (near-identical open/close events), and *cutlery*↔*water/keychain* (metallic, clattering kitchen sounds that co-occur). The brightest diagonals match the highest per-class F1, while the near-empty *light_switch* row confirms it almost never fires confidently — consistent with the imperfect upper bound implied by annotator disagreement (Section 2). Overall the detector is strong on sustained, distinctive events and weak on rare impulsive ones, and its mistakes are dominated by confusions between same-family sounds.

Disclosure of LLM and AI Tool Use

Claude (Anthropic) and ChatGPT were used to assist with Python implementation of the data processing and classification pipelines, LaTeX formatting, and improving writing clarity. All generated code was tested on our dataset and modified where necessary. All reported results were verified by running the experiments and inspecting the outputs. Content was critically reviewed for technical accuracy before inclusion.

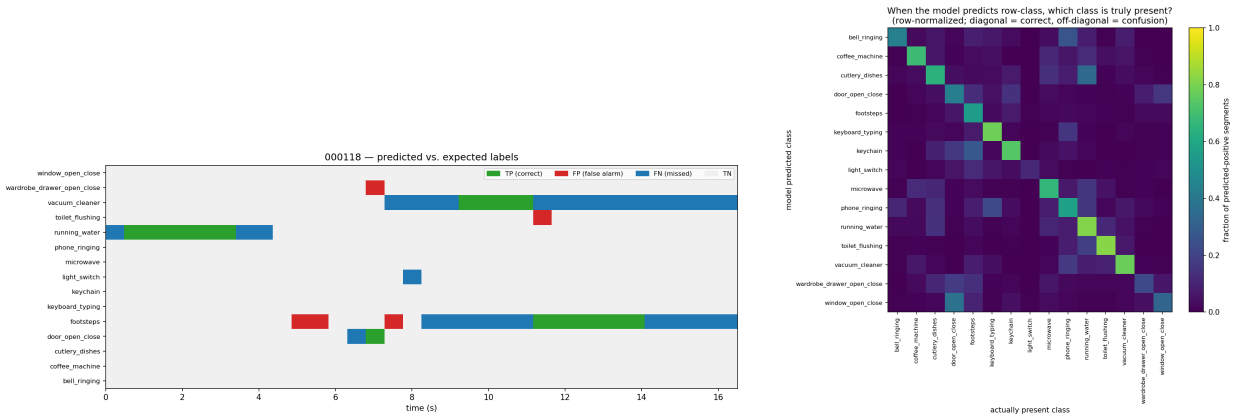

Figure 7: Left: file 000118 – predicted vs. expected labels (same legend as Figure 6); the vacuum cleaner (long blue band) is mostly missed outside its loudest portion. Right: cross-class confusion (row-normalized over predicted-positive segments); off-diagonal mass marks systematic confusions.